



The Guardrail Catalogue

Forty-five techniques for governing AI agents — and what Craft A Bot does with each

The first edition of the catalogue that ships inside Craft A Bot, laid out for reading: each technique described in plain terms, categorised by what it does, rated for maturity, mapped to the threats it answers, and marked — honestly — with the product's own coverage.



EDITION 2026-09 • PENDING REVIEW

How to read this catalogue

This is the first edition of the Guardrail Catalogue that ships inside Craft A Bot: forty-five techniques for governing AI agents, gathered from the industry’s products, the standards bodies and the research literature, and sorted into six categories by what they do. Each entry says what the technique is in plain terms, how mature it is, which threats it answers, and — separately and without inference — what Craft A Bot actually does about it today.

What each entry shows

Name and id	The technique, and the identifier it has in the catalogue’s content (<code>packages/governance/sic/catalogue</code>). Techniques, not vendors: the vendors and papers that ship or propose a technique are its sources.
Sub-category	The finer grouping within the category — an input guardrail, an action control, a monitoring measure, and so on.
Maturity	How established the technique is in practice (see below).
Points	Where in the agent loop the control decides, if it decides at all. A dash means the technique is a measure, a practice or an architecture rather than a decision in the loop.
Protects against	The OWASP threat codes the technique answers: <i>AS/</i> codes from the Top 10 for Agentic Applications (2026 edition), <i>LLM</i> codes from the Top 10 for LLM Applications 2025. A key is at the back.
In Craft A Bot	The coverage status and the note the content carries, as of the 12 September 2026 snapshot, with the work package that landed it. The build refuses a status the code cannot back.
Implemented by	The components or mechanisms in the product, in the code’s own words; file numbers refer to the design notes in <code>docs/design-day2/</code> .
Sources	At least one standard, vendor document, paper or guidance note per entry, with publisher and year.

Coverage status

Coverage says what the product does, not what the technique could do. The five statuses are mutually exclusive and the catalogue build (`checkCatalogue`) refuses an entry whose status does not match what it names — a *shipped* entry that names nothing, a *blueprint* that names an implementation, a *not applicable* with no reason.

SHIPPED	a component or a mechanism exists in Craft A Bot and the register can show what it did
CONNECTABLE	the guard shell meets a vendor's contract and a checkpoint proves it
BESPOKE	a design of record exists, built in part or as content
BLUEPRINT	described and mapped; nothing built
NOT APPLICABLE	said, with the reason

In this edition: **33 shipped**, 0 connectable, **6 bespoke**, **3 blueprint**, **3 not applicable**. All forty-five entries are marked *pending review*: the content awaits a reader's check against its sources, and marking an entry reviewed is a content change with a reader's name on it, never a build step.

Maturity

Widely adopted	in general commercial use; several vendors or standards ship it
Emerging	shipping from some vendors or named in recent guidance; practice still settling
Research	described in papers; little or no product implementation yet

Points in the loop

pre-think	before the bot reasons over its next turn
pre-act	before an action the bot has chosen is carried out
post-act	after an action's result comes back, before the bot reads it
stage-in	at a workflow stage's input boundary
stage-out	at a workflow stage's output boundary
group	at the group chokepoint, across all seats in an episode
egress	on an outbound network call

Contents

Runtime protection

23 ENTRIES

#	TECHNIQUE	SUB-CATEGORY	MATURITY	IN CRAFT A BOT
01	Prompt-injection classifier	Input guardrail	Widely adopted	SHIPPED
02	Jailbreak classifier	Input guardrail	Widely adopted	SHIPPED
03	Hazard and content classifier	Input guardrail	Widely adopted	SHIPPED
04	Policy-conditioned classifier	Input guardrail	Emerging	BLUEPRINT
05	Untrusted-content marking	Input guardrail	Emerging	BESPOKE
06	Indirect-injection defences for tool results	Input guardrail	Emerging	BESPOKE
07	Memory and context-poisoning defences	Input guardrail	Research	BESPOKE
08	PII detection and redaction	Output guardrail	Widely adopted	SHIPPED
09	Domain output rules	Output guardrail	Widely adopted	SHIPPED
10	Output-side judges and hallucination checks	Output guardrail	Widely adopted	SHIPPED
11	Structured-output validation	Output guardrail	Widely adopted	SHIPPED
12	Tool allow and deny lists	Action control	Widely adopted	SHIPPED
13	Policy-as-code decision point	Policy as code	Emerging	SHIPPED
14	Runtime enforcement rules	Policy as code	Research	SHIPPED
15	Least-privilege scopes with recorded elevation	Policy as code	Research	BLUEPRINT
16	Step and token budgets	Action control	Widely adopted	SHIPPED
17	Loop and no-progress detection	Action control	Widely adopted	SHIPPED
18	Rate limits	Action control	Widely adopted	NOT APPLICABLE
19	Information-flow control and taint tracking	Information flow	Research	BESPOKE
20	Monitor agents and circuit breakers	Monitoring	Emerging	SHIPPED
21	Behavioural drift and anomaly detection	Monitoring	Emerging	SHIPPED
22	Evidence tracing and execution provenance	Monitoring	Widely adopted	SHIPPED
23	Stage-boundary guards	Policy as code	Emerging	SHIPPED

Secure by design

3 ENTRIES

#	TECHNIQUE	SUB-CATEGORY	MATURITY	IN CRAFT A BOT
24	Privilege separation and dual-LLM patterns	Privilege separation	Emerging	BESPOKE
25	Formal verification of agent policies	Formal verification	Research	BLUEPRINT
26	Guardrail frameworks	Framework	Widely adopted	NOT APPLICAB

Identity and access

3 ENTRIES

#	TECHNIQUE	SUB-CATEGORY	MATURITY	IN CRAFT A BOT
27	Agent identity, delegation chains and attestation	Identity	Emerging	SHIPPED
28	Credential hygiene	Credentials	Widely adopted	SHIPPED
29	Inter-agent message authentication	Multi-agent	Emerging	BESPOKE

Component hardening

3 ENTRIES

#	TECHNIQUE	SUB-CATEGORY	MATURITY	IN CRAFT A BOT
30	Sandboxed tool and code execution	Isolation	Widely adopted	NOT APPLICAB
31	Egress control	Isolation	Widely adopted	SHIPPED
32	Pack, tool and cassette integrity	Supply chain	Emerging	SHIPPED

Evaluation and assurance

7 ENTRIES

#	TECHNIQUE	SUB-CATEGORY	MATURITY	IN CRAFT A BOT
33	Evaluation harnesses and policy-compliance benchmarks	Evaluation	Widely adopted	SHIPPED
34	Automated red-teaming	Red-teaming	Emerging	SHIPPED
35	Safety cases and transparency artefacts	Assurance	Emerging	SHIPPED
36	Incident reporting	Assurance	Widely adopted	SHIPPED
37	Control-effectiveness measurement	Assurance	Emerging	SHIPPED
38	Fairness and parity measurement	Evaluation	Widely adopted	SHIPPED
39	Governance frameworks and management systems	Framework	Widely adopted	SHIPPED

Human oversight

6 ENTRIES

#	TECHNIQUE	SUB-CATEGORY	MATURITY	IN CRAFT A BOT
40	Risk-tiered approval	Human in the loop	Widely adopted	SHIPPED
41	Four-eyes and decision-rights ceilings	Human in the loop	Widely adopted	SHIPPED
42	Kill switch and safe-mode degradation	Interruptibility	Widely adopted	SHIPPED
43	Levels of autonomy	Autonomy	Emerging	SHIPPED
44	Confirmation fatigue	Human in the loop	Research	SHIPPED
45	Explainability to end users	Explainability	Widely adopted	SHIPPED

Runtime protection

Controls that act while the bot is running — on what it reads, what it is about to do, what it says and what comes back from its tools. This is the largest group because it is where most of the industry's products sit: classifiers in front of the model, filters behind it, rules over its actions and monitors over the run. The sub-categories say where in the loop each control decides.

Input guardrail 7 · Output guardrail 4 · Action control 4 · Policy as code 4 · Information flow 1 · Monitoring 3

01 Prompt-injection classifier

SHIPPED

prompt-injection-classifier

SUB-CATEGORY Input guardrail MATURITY Widely adopted POINTS pre-think, pre-act

A classifier over what the bot is about to read or do that flags an injected instruction.

Prompt injection is text that arrives as data — a customer message, a retrieved document, a tool result — but is written to read as an instruction to the bot (“ignore your rules and move the balance”). A prompt-injection classifier is a small model or a hosted service that scores each piece of incoming text for that pattern before the bot reasons over it or acts on it, and flags or blocks what it finds. It sits in front of every other control: if an injected instruction gets through, the rest of the stack is defending a bot that has already been redirected.

PROTECTS AGAINST ASI01 Agent goal hijack; LLM01 Prompt injection

IN CRAFT A BOT SHIPPED Five services through the shell, each with an offline stand-in; Lakera and Bedrock are harness-only connections (WP99) with their live checkpoints pending. WP42

IMPLEMENTED BY guard-local/prompt-guard; geap/model-armor; azure-content-safety/content-safety; lakera-guard/guard; bedrock-guardrails/apply-guardrail

- SOURCES
- Meta, *Llama Guard 4 and Prompt Guard 2 model cards* (2025).
 - Google Cloud, *Model Armor documentation (filter version 3)* (2026).
 - Microsoft, *Azure AI Content Safety — Prompt Shields and harm categories* (2025).
 - Lakera, *Lakera Guard documentation* (2025).
 - AWS, *Amazon Bedrock Guardrails (prompt-attack filter, automated reasoning checks)* (2025).
 - OWASP GenAI Security Project, *OWASP Top 10 for Agentic Applications (2026 edition)* (2025).
 - OWASP GenAI Security Project, *OWASP Top 10 for LLM Applications 2025* (2025).

02 Jailbreak classifier

SHIPPED

jailbreak-classifier

SUB-CATEGORY Input guardrail **MATURITY** Widely adopted **POINTS** pre-think

A classifier trained to spot attempts to override the model's own rules.

A jailbreak is an attempt to talk a model out of its own rules — a role-play framing, a fictional “developer mode”, encoded or fragmented instructions, sustained pressure over many turns. A jailbreak classifier is trained on those attempts and scores the user's turn for them before the model thinks. It overlaps with the injection classifier but targets the person talking to the bot rather than content smuggled in from elsewhere.

PROTECTS AGAINST ASI01 Agent goal hijack; LLM01 Prompt injection**IN CRAFT A BOT** **SHIPPED** Prompt Guard 2 and Model Armor's jailbreak detection; constitutional classifiers are a lab's own deployment, not a service the shell can reach. WP42**IMPLEMENTED BY** guard-local/prompt-guard ; geap/model-armor**SOURCES**

- Meta, *Llama Guard 4 and Prompt Guard 2 model cards* (2025).
- Google Cloud, *Model Armor documentation (filter version 3)* (2026).
- Anthropic, *Constitutional Classifiers: defending against universal jailbreaks* (2025).
- Microsoft, *Azure AI Content Safety — Prompt Shields and harm categories* (2025).

03 Hazard and content classifier

SHIPPED

hazard-classifier

SUB-CATEGORY Input guardrail **MATURITY** Widely adopted **POINTS** pre-think, pre-act, post-act

A classifier over a taxonomy of harms (MLCommons, Azure severities) applied to prompts and responses.

A classifier that places text in a taxonomy of harms — violence, self-harm, sexual content, hate, and categories a bank cares about such as facilitating fraud — and rates its severity. Applied on the way in and on the way out, so a bot neither reasons over nor says something in a category the deployer has excluded. The taxonomies come from MLCommons and the vendors; the thresholds are the deployer's.

PROTECTS AGAINST LLM05 Improper output handling; ASI09 Human-agent trust exploitation**IN CRAFT A BOT** **SHIPPED** Llama Guard 4 over Ollama, Azure Content Safety, and Model Armor's responsible-AI filters. WP42**IMPLEMENTED BY** guard-local/llama-guard ; azure-content-safety/content-safety ; geap/model-armor**SOURCES**

- Meta, *Llama Guard 4 and Prompt Guard 2 model cards* (2025).
- Microsoft, *Azure AI Content Safety — Prompt Shields and harm categories* (2025).
- OpenAI, *Moderation API and gpt-oss-safeguard* (2025).
- Google and IBM, *ShieldGemma and Granite Guardian model cards* (2024).
- Google Cloud, *Model Armor documentation (filter version 3)* (2026).

04

Policy-conditioned classifier

BLUEPRINT

policy-conditioned-classifier

SUB-CATEGORY

Input guardrail

MATURITY

Emerging

POINTS

pre-think

pre-act

A safety reasoner that takes the deployer’s written policy at inference time and classifies against it.

Most classifiers carry a fixed taxonomy chosen by the vendor. A policy-conditioned classifier is given the deployer’s own written policy at inference time and reasons about whether a text complies with it — so a bank’s rulebook, not a vendor’s list, is what the check is made against. This is new (OpenAI’s gpt-oss-safeguard, 2025) and untested at scale.

PROTECTS AGAINST

ASI01

Agent goal hijack;

ASI09

Human-agent trust exploitation

IN CRAFT A BOT

BLUEPRINT

Day 6 names a component over any cartridge with the bank’s rulebook as the policy; nothing built.

IMPLEMENTED BY

—

SOURCES

- OpenAI, *Moderation API and gpt-oss-safeguard* (2025).

05

Untrusted-content marking

BESPOKE

untrusted-content-marking

SUB-CATEGORY

Input guardrail

MATURITY

Emerging

POINTS

pre-think

post-act

Spotlighting, instruction hierarchy or a control/data split so retrieved text cannot pose as an instruction.

A language model cannot by itself tell an instruction from a quotation. Marking techniques make the boundary explicit: spotlighting wraps retrieved text in delimiters and tells the model it is data; an instruction hierarchy trains the model to rank the operator’s instructions above the user’s and the user’s above any content; a control/data split (CaMeL) keeps untrusted text out of the planning model altogether. The aim is that a retrieved page or a tool result can inform the bot but never command it.

PROTECTS AGAINST

ASI01

Agent goal hijack;

ASI06

Memory and context poisoning;

LLM01

Prompt injection

IN CRAFT A BOT

BESPOKE

The desk brief separates the records from the instructions; tool results and line answers are not yet marked — Day 6 names an untrusted-content component at post-act.

IMPLEMENTED BY

desk brief: records apart from instructions (43-DESK-WORLDS.md)

SOURCES

- Microsoft Research, *Defending against indirect prompt injection with spotlighting* (2024).
- OpenAI, *The Instruction Hierarchy: training LLMs to prioritise privileged instructions* (2024).
- Google DeepMind, *Defeating prompt injections by design (CaMeL)* (2025).

06

Indirect-injection defences for tool results

BESPOKE

indirect-injection-defence

SUB-CATEGORY

Input guardrail

MATURITY

Emerging

POINTS

post-act

Treating tool output, retrieved documents and MCP descriptions as attack surface — poisoned tools, confused deputies.

Indirect injection arrives not from the person talking to the bot but through what the bot fetches: a poisoned factsheet, a note in the CRM, a doctored payslip, a tool whose description carries hidden instructions. These defences treat every tool result, retrieved document and tool description as attack surface — scanning them, checking tool descriptions against a registry, and testing the bot under such attacks so that its behaviour when it meets one is known rather than hoped for.

- PROTECTS AGAINST
- ASI02
- Tool misuse and exploitation;
- ASI04
- Agentic supply-chain vulnerabilities;
- LLM01
- Prompt injection
- IN CRAFT A BOT
- BESPOKE
- Shipped as scenarios the decks run under pressure; a tool-description integrity check on the registry is a blueprint.
- IMPLEMENTED BY
- the poisoned factsheet, the CRM note and the doctored payslip decks (19-... #38)
- SOURCES
- OWASP GenAI Security Project, *OWASP Top 10 for Agentic Applications (2026 edition)* (2025).
 - Model Context Protocol, *MCP security best practices* (2025).
 - MITRE, *MITRE ATLAS (agentic techniques, 2025–26 updates)* (2025).

07

Memory and context-poisoning defences

BESPOKE

memory-provenance

SUB-CATEGORY

Input guardrail

MATURITY

Research

POINTS

pre-think

Provenance tags on what a bot remembers, quarantine of untrusted memory, refusal to reason over it.

A bot that remembers across turns or across cases can be poisoned once and misled for good. Memory-provenance defences tag every memory with where it came from and how trusted that source is, quarantine memory that was written from untrusted content, and refuse to let the bot reason over a memory whose provenance is unknown. Still largely a research topic; the attacks are well documented.

- PROTECTS AGAINST
- ASI06
- Memory and context poisoning
- IN CRAFT A BOT
- BESPOKE
- Every notebook write is on the trace; no provenance tag yet — Day 6 names a memory-provenance component and a card that refuses a think over untrusted memory.
- IMPLEMENTED BY
- memory.updated on the trace (02-AGENT-MODEL.md §7)
- SOURCES
- arXiv, *Memory-poisoning attacks on LLM agents (2025–26 research)* (2025).
 - OWASP GenAI Security Project, *OWASP Top 10 for Agentic Applications (2026 edition)* (2025).

08 PII detection and redaction

SHIPPED

pii-redaction

SUB-CATEGORY Output guardrail **MATURITY** Widely adopted **POINTS** pre-act

Finding personal identifiers in what the bot is about to say and rewriting the line before it goes out.

Before a line leaves the bot it is scanned for personal identifiers — names, account and card numbers, addresses, dates of birth — and the identifiers are masked or removed, so the outgoing text is safe to show even when the model was careless. Output-side redaction is a last line of defence for data protection, not a substitute for keeping the data out of the model’s context in the first place.

PROTECTS AGAINST LLM02 Sensitive information disclosure; ASI09 Human-agent trust exploitation

IN CRAFT A BOT SHIPPED Model Armor’s Sensitive Data Protection through the shell, its redaction applied to the outgoing line since WP96; the PII-contained evaluator reads the redaction. WP96

IMPLEMENTED BY geap/model-armor; the redact verdict applied to say (85-COMPONENTS.md §4); fs-advice/pii-contained

- SOURCES**
- Microsoft, *Presidio — data protection and de-identification* (2024).
 - Google Cloud, *Model Armor documentation (filter version 3)* (2026).
 - Guardrails AI, *Guardrails AI validators and hub* (2025).

09 Domain output rules

SHIPPED

domain-output-rules

SUB-CATEGORY Output guardrail **MATURITY** Widely adopted **POINTS** pre-act

The bank’s own rulebook as output filters: no guarantee language, no tipping off, plain English, promotions wording.

The bank’s own rules for what may be said, applied as filters on the bot’s outgoing lines: no language that reads as a guarantee, no tipping off a customer about a suspicion, plain English, the required wording on a financial promotion. The rules are the regulator’s (COBS, the Consumer Duty, the Proceeds of Crime Act) written down as checks rather than left to the model’s discretion.

PROTECTS AGAINST ASI09 Human-agent trust exploitation

IN CRAFT A BOT SHIPPED Every desk’s cards, compiled as components. WP60

IMPLEMENTED BY governance/policy-card; the desks’ policy cards (49-, 51-, 52-, 61-...)

- SOURCES**
- Financial Conduct Authority, *FCA Handbook — COBS 4 (financial promotions), COBS 9 (suitability), PRIN 2A (Consumer Duty)* (2023).
 - UK Parliament, *Proceeds of Crime Act 2002, s.333A (tipping off)* (2002).

10 Output-side judges and hallucination checks

SHIPPED

llm-as-judge

SUB-CATEGORY Output guardrail **MATURITY** Widely adopted **POINTS** post-act, stage-out, group

A second model or a validator judging an output for hazard, faithfulness or hallucination.

A second model, or a validator, reads what the bot produced and judges it: is it hazardous, is it faithful to the records it was given, does it assert something the sources do not support? A failing judgment can block the line, stop the run or raise a finding. Judges catch what pattern rules cannot, at the cost of a second model's own fallibility — so a judge is measured like any other control.

PROTECTS AGAINST LLM09 Misinformation; ASI09 Human-agent trust exploitation

IN CRAFT A BOT **SHIPPED** The rubric and hosted evaluators as judges with offline stand-ins; the breaker fits a judge at the chokepoint or a stage boundary. Bedrock's automated-reasoning checks are a WP99 connection candidate, recorded research-grade for a bank's rulebook. WP43

IMPLEMENTED BY monitor/evaluator-breaker; the rubric evaluators (31-...); geap/eval/* (39-...)

- SOURCES**
- Guardrails AI, *Guardrails AI validators and hub* (2025).
 - AWS, *Amazon Bedrock Guardrails (prompt-attack filter, automated reasoning checks)* (2025).
 - UK AI Security Institute, *Inspect evaluation framework* (2024).

11 Structured-output validation

SHIPPED

structured-output-validation

SUB-CATEGORY Output guardrail **MATURITY** Widely adopted **POINTS** stage-out

Every tool call and stage output validated against a schema before anything reads it.

Every tool call the bot makes and every output a workflow stage produces is checked against a declared schema before anything downstream reads it. A malformed or unexpected structure is refused at the boundary, which stops a large class of failures — and injected content hiding in a free-text field — from propagating into the next step.

PROTECTS AGAINST LLM05 Improper output handling; ASI05 Unexpected code execution

IN CRAFT A BOT **SHIPPED** A mechanism of the workflow runtime, not a component: every stage's input and output against its JSON schema. WP79

IMPLEMENTED BY validateAgainst in @craftabot/workflow (69-WORKFLOWS.md §5)

- SOURCES**
- Guardrails AI, *Guardrails AI validators and hub* (2025).
 - OpenAI, *Agents SDK guardrails and AgentKit* (2025).

12

Tool allow and deny lists

SHIPPED

tool-allow-deny

SUB-CATEGORY

Action control

MATURITY

Widely adopted

POINTS

pre-act

What a bot may call, by name and by argument, decided before the call runs.

A declared list of what a bot may and may not call, checked by tool name and by argument before the call runs. A deny list blocks named actions outright; an allow list permits only the named ones; argument rules constrain, for example, which accounts or what amounts a payment tool may be given. The simplest expression of least privilege for an agent.

PROTECTS AGAINST

ASI02

Tool misuse and exploitation;

ASI05

Unexpected code execution;

LLM06

Excessive agency

IN CRAFT A BOT

SHIPPED

The blocklist and the card leaves over arguments (33-...). WP0

IMPLEMENTED BY

governance/action-blocklist; governance/policy-card

SOURCES

- Anthropic, *Claude Code permissions and sandboxing* (2025).
- OpenAI, *Agents SDK guardrails and AgentKit* (2025).
- Vercel, *AI SDK — policy-based tool approvals* (2025).

13

Policy-as-code decision point

SHIPPED

policy-decision-point

SUB-CATEGORY

Policy as code

MATURITY

Emerging

POINTS

pre-act, stage-in, stage-out

Externalising “may this bot do this?” to a policy engine evaluated per action.

Instead of scattering “may this bot do this?” across the code, the question is sent to a policy engine (Open Policy Agent with Rego, or Cedar) that evaluates written policy against the action, its arguments and the context, and returns a decision. Policy is then versioned, tested and reviewed on its own, apart from the bot, and the same engine can answer for a person or a rule as well as an agent.

PROTECTS AGAINST

ASI02

Tool misuse and exploitation;

ASI03

Identity and privilege abuse

IN CRAFT A BOT

SHIPPED

OPA through the shell with its live checkpoint taken; Cedar is a blueprint connection. WP45

IMPLEMENTED BY

pdp-opa/opa; PredicateExpr v2 as the built-in engine (33-...)

SOURCES

- CNCF, *Open Policy Agent and Rego* (2024).
- AWS, *Cedar policy language; Bedrock AgentCore gateway policies* (2025).
- Microsoft, *Agent Governance Toolkit — OPA, Rego and Cedar policies for agents* (2025).

14 Runtime enforcement rules

SHIPPED

runtime-enforcement-dsl

SUB-CATEGORY Policy as code **MATURITY** Research **POINTS** pre-think, pre-act, post-act

Trigger → predicate → enforcement rules intercepting the agent loop (AgentSpec's shape).

A small rule language for the agent loop: on a trigger (before a think, before an action, after a result) evaluate a predicate over the current state, and apply an enforcement — allow, block, stop the run, ask a person. AgentSpec is the research statement of this shape; Progent applies it to privilege. Craft A Bot's policy cards are the same three parts.

PROTECTS AGAINST ASI01 Agent goal hijack; ASI02 Tool misuse and exploitation

IN CRAFT A BOT SHIPPED Policy cards are the AgentSpec shape: a hook, a predicate, a disposition. WP22

IMPLEMENTED BY governance/policy-card

SOURCES

- ICSE (arXiv 2503.18666), *AgentSpec: customizable runtime enforcement for safe and reliable LLM agents* (2026).
- arXiv 2504.11703, *Progent: programmable privilege control for LLM agents* (2025).

15 Least-privilege scopes with recorded elevation

BLUEPRINT

privilege-scopes

SUB-CATEGORY Policy as code **MATURITY** Research **POINTS** pre-act

A bot started with minimal grants, every elevation request recorded and decided.

A bot starts with the least privilege its task needs. When it needs more it asks; the request and the decision are recorded, and the elevation expires. The record is what makes an audit possible — every grant a bot ever held is explained by a request someone decided.

PROTECTS AGAINST ASI03 Identity and privilege abuse

IN CRAFT A BOT BLUEPRINT Day 6 names a privilege-scopes component (19-... #15); the Connector's scopes are the nearest thing today and are not elevation.

IMPLEMENTED BY —

SOURCES

- arXiv 2504.11703, *Progent: programmable privilege control for LLM agents* (2025).
- OWASP GenAI Security Project, *Agentic AI — Threats and Mitigations* (2025).

16 Step and token budgets

SHIPPED

budget-cap

SUB-CATEGORY Action control **MATURITY** Widely adopted **POINTS** pre-think

A hard cap on turns and tokens so a runaway bot ends the run rather than the bill.

Hard ceilings on the turns and the tokens a run may consume. When a cap is reached the run ends with a named outcome rather than looping until the bill, or the risk, has grown. The simplest runtime control and among the most effective against a runaway agent.

PROTECTS AGAINST ASI08 Cascading failures

IN CRAFT A BOT SHIPPED OUT_OF_STEPS is the loop score. WP0

IMPLEMENTED BY governance/step-budget ; governance/token-budget

SOURCES

- LangChain, *LangGraph recursion limits and time-travel debugging* (2025).
- OpenAI, *Agents SDK guardrails and AgentKit* (2025).

17 Loop and no-progress detection

SHIPPED

loop-detection

SUB-CATEGORY Action control MATURITY Widely adopted POINTS pre-act

Refusing the same non-progress move proposed too many times running.

A bot that keeps proposing the same non-progress move — the same failed call, the same question, the same edit — is stopped after a set number of repetitions. Progress is judged by a predicate the world supplies, so the detector knows what counts as forward motion in that domain rather than guessing from the text.

PROTECTS AGAINST ASI08 Cascading failures

IN CRAFT A BOT SHIPPED The loop-breaker with the world's progress predicate; a broader no-progress detector over repeated identical calls is a blueprint (19-... #7). WP30

IMPLEMENTED BY governance/no-repetition

SOURCES • LangChain, LangGraph recursion limits and time-travel debugging (2025).

18 Rate limits

NOT APPLICABLE

rate-limit

SUB-CATEGORY Action control MATURITY Widely adopted POINTS —

Calls per minute or per account, bounded at the gateway.

A bound on calls per minute or per account, enforced at the gateway rather than inside the bot, so that a fault or an attack cannot turn into thousands of requests. Standard in production; not applicable in a simulator, where a run has no wall clock a rate could be measured against.

PROTECTS AGAINST ASI08 Cascading failures

IN CRAFT A BOT NOT APPLICABLE Not applicable to a simulator: a run has no wall clock a rate could be measured against — the budgets are the bound, and a provider's own limit reaches the trace as provider.retryed.

IMPLEMENTED BY —

SOURCES • OpenAI, Agents SDK guardrails and AgentKit (2025).

19 Information-flow control and taint tracking

BESPOKE

information-flow-control

SUB-CATEGORY Information flow **MATURITY** Research **POINTS** pre-act, post-act

Labels on data that follow it through the bot so a secret cannot reach an egress or a line the purpose forbids.

Labels attached to data — a classification, a purpose — follow that data as the bot works, so a secret read for one purpose cannot flow to an outbound call or into a line the purpose forbids. Full taint tracking through a model's reasoning is research (FIDES, CaMeL); classifying every record and gating every line by purpose is the practical part that can be built today.

PROTECTS AGAINST ASI06 Memory and context poisoning; LLM02 Sensitive information disclosure

IN CRAFT A BOT **BESPOKE** Every record is classified and every line purpose-gated; no taint through the bot's reasoning yet — Day 6 names a taint component and a taint-reaches card leaf.

IMPLEMENTED BY classification on every record and purpose-gating on every line (48-FS-BANK.md, tenet 13)

SOURCES

- arXiv, *FIDES: information-flow control for LLM agents* (2025).
- Google DeepMind, *Defeating prompt injections by design (CaMeL)* (2025).
- Simon Willison, *The lethal trifecta for AI agents; the dual-LLM pattern* (2025).

20 Monitor agents and circuit breakers

SHIPPED

monitor-agent

SUB-CATEGORY Monitoring **MATURITY** Emerging **POINTS** group, stage-out, post-act

A second seat or an evaluator watching the run and stopping it on a named failure.

A second seat — a monitor bot or an evaluator — watches the run as it happens and can stop it on a named failure. A circuit breaker is the mechanical form: an evaluator fitted at a chokepoint that trips when its verdict fails. SHADE-Arena studies how well a monitoring model catches an agent that is quietly pursuing a different goal.

PROTECTS AGAINST ASI08 Cascading failures; ASI10 Rogue agents

IN CRAFT A BOT **SHIPPED** The breaker at the chokepoint since WP64 and at a stage boundary since WP95; the Watchbot is a brick. WP48

IMPLEMENTED BY monitor/evaluator-breaker; the group Watchbot and the Compliance Watchbot (36-, 56-...)

SOURCES

- Anthropic, *SHADE-Arena: evaluating sabotage and monitoring in LLM agents* (2025).
- OWASP GenAI Security Project, *OWASP Top 10 for Agentic Applications (2026 edition)* (2025).

21 Behavioural drift and anomaly detection

SHIPPED

behavioural-drift

SUB-CATEGORY Monitoring MATURITY Emerging POINTS —

A statistic over the run series that flags a change in what the bot does before a person would.

A statistic over the series of runs — Page–Hinkley change detection is the classic — that flags when what the bot does has shifted (more refusals, more escalations, more incidents) before a person reading periodic reports would notice. Drift detection is a measure over telemetry rather than a decision in the loop, and needs a stored series to work on.

PROTECTS AGAINST ASI08 Cascading failures

IN CRAFT A BOT SHIPPED Page–Hinkley and the Monitor over the stored series; not a component, a fold. WP49

IMPLEMENTED BY @craftabot/metrics driftIn (68-...); the Monitor (75-...)

- SOURCES
- E. S. Page, Biometrika; agent observability guidance, *Page–Hinkley and behavioural drift detection on agent telemetry* (1954).
 - OpenTelemetry, *OpenTelemetry semantic conventions for generative AI* (2025).

22 Evidence tracing and execution provenance

SHIPPED

execution-provenance

SUB-CATEGORY Monitoring MATURITY Widely adopted POINTS —

Every prompt, decision, action and verdict as a typed event, digested, with who was behind it.

Every prompt, decision, action and verdict is a typed event on a trace, digested so it cannot be altered unnoticed, carrying the principal that was behind it. This is what turns a bot’s behaviour from an anecdote into evidence a reviewer can replay, and what the EU AI Act’s record-keeping articles ask for.

PROTECTS AGAINST ASI10 Rogue agents; ASI03 Identity and privilege abuse

IN CRAFT A BOT SHIPPED Hard rule 3: everything observable arrives as an event. WP0

IMPLEMENTED BY the trace and its digest (07-...); principal and attestation (55-...); the OTel mapping (35-...)

- SOURCES
- OpenTelemetry, *OpenTelemetry semantic conventions for generative AI* (2025).
 - Arize, *OpenInference — GUARDRAIL and agent spans* (2025).
 - European Union, *Regulation (EU) 2024/1689 (the AI Act), Articles 9, 12, 13 and 14* (2024).

23 Stage-boundary guards

SHIPPED

stage-boundary-guard

SUB-CATEGORY Policy as code MATURITY Emerging POINTS stage-in, stage-out

A guard that decides at a stage’s input or output whatever the executor – a rule, a person or a bot.

In a multi-stage workflow the guard decides at a stage’s input or output — whatever executed the stage: a rule, a person or a bot. Guarding the boundary rather than the executor makes the check independent of who did the work, and lets a human stage be held to the same output contract as an automated one.

PROTECTS AGAINST ASI02 Tool misuse and exploitation; ASI08 Cascading failures

IN CRAFT A BOT SHIPPED The workflow runtime’s boundary chain (69-... §10). WP95

IMPLEMENTED BY governance/policy-card; monitor/evaluator-breaker; geap/model-armor

- SOURCES
- ICSE (arXiv 2503.18666), *AgentSpec: customizable runtime enforcement for safe and reliable LLM agents* (2026).
 - OWASP GenAI Security Project, *OWASP Top 10 for Agentic Applications (2026 edition)* (2025).

Secure by design

Architectural choices that remove a class of attack rather than detect it: keeping untrusted text away from the model that holds the tools, proving a policy rather than checking each action, and the frameworks that package rails around a model.

Privilege separation 1 · Formal verification 1 · Framework 1

24 Privilege separation and dual-LLM patterns

BESPOKE

privilege-separation

SUB-CATEGORY Privilege separation MATURITY Emerging POINTS group

A privileged planner that never reads untrusted content and a quarantined reader that never acts.

The dual-LLM pattern: a privileged planner that never reads untrusted content, and a quarantined reader that reads it but cannot act. Untrusted text therefore never reaches the model that holds the tools. CaMeL formalises this with a capability system over what the reader may hand back; AirGapAgent applies the same idea to private data. An architecture, not a filter — it removes the attack instead of detecting it.

PROTECTS AGAINST ASI01 Agent goal hijack; ASI07 Insecure inter-agent communication

IN CRAFT A BOT **BESPOKE** The Watchbot is a second seat with its own brain; a quarantined reader seat is a blueprint configuration of the episode.

IMPLEMENTED BY the two-seat episode (46-, 56-...)

- SOURCES
- Simon Willison, *The lethal trifecta for AI agents; the dual-LLM pattern* (2025).
 - Google DeepMind, *Defeating prompt injections by design (CaMeL)* (2025).
 - arXiv, *AirGapAgent: protecting privacy with context hijacking* (2024).

25

Formal verification of agent policies

BLUEPRINT

formal-verification

SUB-CATEGORY

Formal verification

MATURITY

Research

POINTS

—

Proving a policy holds over the agent’s possible actions rather than checking each one.

Rather than checking each action as it comes, prove that a policy holds over every action a bot could take — for example that no path leads to a payment without an approval. Research today (ShieldAgent); Amazon Bedrock’s automated-reasoning checks are the nearest commercial form, and are aimed at factual policy compliance rather than agent actions.

PROTECTS AGAINST

ASI02

Tool misuse and exploitation

IN CRAFT A BOT

BLUEPRINT

Recorded as research; Bedrock’s automated-reasoning checks would be a policy-engine connection.

IMPLEMENTED BY

—

SOURCES

- arXiv, *ShieldAgent: shielding agents via verifiable safety policy reasoning* (2025).
- AWS, *Amazon Bedrock Guardrails (prompt-attack filter, automated reasoning checks)* (2025).

26

Guardrail frameworks

NOT APPLICABLE

guardrail-framework

SUB-CATEGORY

Framework

MATURITY

Widely adopted

POINTS

pre-think

pre-act

post-act

Programmable rails (NeMo, Guardrails AI) wrapping a model’s input and output.

Programmable rails — NVIDIA NeMo Guardrails, Guardrails AI, the OpenAI Agents SDK — that wrap a model’s input and output with validators, dialogue flows and hooks. Useful as a dependency for a team building from scratch; Craft A Bot has its own guard shell and component contract, and treats a vendor’s validators as services it can reach rather than as rails it runs inside.

PROTECTS AGAINST

LLM01

Prompt injection;

LLM05

Improper output handling

IN CRAFT A BOT

NOT APPLICABLE

Not applicable as a dependency: the guard shell (29-...) and the component contract (85-...) are the product’s own framework; a vendor framework’s validators reach it as a service, not as rails.

IMPLEMENTED BY

—

SOURCES

- NVIDIA, *NeMo Guardrails* (2024).
- Guardrails AI, *Guardrails AI validators and hub* (2025).
- OpenAI, *Agents SDK guardrails and AgentKit* (2025).

Identity and access

Who the bot is, whose authority it acts under, and how it proves it — to a directory, to another agent, and on the trace. The same questions a bank asks of a member of staff, asked of an agent.

Identity 1 · Credentials 1 · Multi-agent 1

27 Agent identity, delegation chains and attestation

SHIPPED

agent-identity

SUB-CATEGORY Identity MATURITY Emerging POINTS —

A bot as an auditable principal acting on behalf of a person, every action attested with what let it through.

A bot as an auditable principal: it has an identity of its own, acts on behalf of a named person, and every action is attested with what let it through — the grant, the approval, the delegation chain. Microsoft Entra Agent ID, Okta's Cross App Access and SPIFFE are the industry's moves to give agents identities a directory can manage and revoke.

PROTECTS AGAINST ASI03 Identity and privilege abuse; ASI10 Rogue agents

IN CRAFT A BOT SHIPPED On the trace when the host names one; no directory integration. WP65

IMPLEMENTED BY principal, onBehalfOf and attestation (55-PRINCIPAL.md)

- SOURCES
- Microsoft, *Microsoft Entra Agent ID* (2025).
 - Okta, *Cross App Access (XAA) and OAuth extensions for agents* (2025).
 - CNCF, *SPIFFE/SPIRE workload identity* (2024).

28 Credential hygiene

SHIPPED

credential-hygiene

SUB-CATEGORY Credentials MATURITY Widely adopted POINTS —

Keys in a vault, read at call time, never in a file, a trace or a URL, with a test that proves it.

Keys live in a vault and are read at call time; they are never written to a file, never appear on the trace, never travel in a URL — and a test proves that no key can leak into an artefact. Basic, and the control most often found broken when agents are wired to real systems.

PROTECTS AGAINST ASI03 Identity and privilege abuse; LLM02 Sensitive information disclosure

IN CRAFT A BOT SHIPPED Hard rule 2. WP0

IMPLEMENTED BY the vault (cab.keys.v1); the key-leak test; timed vault entries (26-... §6.6)

- SOURCES
- CISA and partner agencies, *Multi-agency guidance on securing agentic AI* (2026).
 - Model Context Protocol, *MCP security best practices* (2025).

29 Inter-agent message authentication

BESPOKE

inter-agent-authentication

SUB-CATEGORY Multi-agent MATURITY Emerging POINTS pre-think

A message from another agent verified before it is trusted.

When bots talk to bots, a message from another agent is verified — who sent it, that it was not altered in transit — before it is trusted. The attack is an impostor agent on the channel (the party-line scenario); the Agent2Agent protocol is where the industry is placing the authentication.

PROTECTS AGAINST ASI07 Insecure inter-agent communication

IN CRAFT A BOT BESPOKE Shipped as a scenario a card can catch; authentication itself is a blueprint.

IMPLEMENTED BY the party-line scenario (starter/party-line)

- SOURCES
- Google and the Linux Foundation, *Agent2Agent (A2A) protocol* (2025).
 - OWASP GenAI Security Project, *OWASP Top 10 for Agentic Applications (2026 edition)* (2025).

Component hardening

Containing what a bot can reach: where its code runs, where its network calls may go, and whether the parts it is built from are what they claim to be.

Isolation 2 · Supply chain 1

30 Sandboxed tool and code execution

NOT APPLICABLE

sandboxed-execution

SUB-CATEGORY Isolation MATURITY Widely adopted POINTS —

Running agent-generated code in a disposable microVM or userspace kernel with no ambient credentials.

Code a bot writes or runs executes in a disposable microVM or a userspace kernel with no ambient credentials, so that a mistake or an injection cannot reach the host or the network. Standard for coding agents (E2B, Daytona, Claude Code's sandbox); not applicable in Craft A Bot, whose bots perform declared actions on a simulated world and run no code.

PROTECTS AGAINST ASI05 Unexpected code execution

IN CRAFT A BOT NOT APPLICABLE Not applicable: the world is the sandbox — a bot performs declared actions on a simulated world and runs no code; recorded so the claim is not implied.

IMPLEMENTED BY —

SOURCES

- AWS, Google and the sandbox vendors, *Firecracker microVMs and gVisor for agent code execution (E2B, Daytona)* (2025).
- Anthropic, *Claude Code permissions and sandboxing* (2025).

31 Egress control

SHIPPED

egress-control

SUB-CATEGORY Isolation MATURITY Widely adopted POINTS egress

Every outbound call from a bot's bricks and provider allowed only to a declared host, or refused and recorded.

Every outbound network call from a bot's tools and its model provider is allowed only to a declared host; anything else is refused and the refusal is recorded. This closes the exfiltration leg of the "lethal trifecta" — private data, untrusted content, and a way to send data out — and is the control the multi-agency guidance puts first.

PROTECTS AGAINST ASI04 Agentic supply-chain vulnerabilities; LLM02 Sensitive information disclosure

IN CRAFT A BOT SHIPPED The session's fetch guard as two components; a refusal is an error on the trace. WP41

IMPLEMENTED BY governance/egress-declared ; governance/egress-none

SOURCES

- Anthropic, *Claude Code permissions and sandboxing* (2025).
- CISA and partner agencies, *Multi-agency guidance on securing agentic AI* (2026).

32

Pack, tool and cassette integrity

SHIPPED

supply-chain-integrity

SUB-CATEGORY

Supply chain

MATURITY

Emerging

POINTS

—

What a bot is built from — packs, tools, recorded lines — declared, versioned and digested.

What a bot is built from — packs, tools, models, recorded lines — is declared, versioned and digested, so a change is visible and a tampered component is refused at load. The AI counterpart of a software bill of materials (CycloneDX's AI/ML BOM) and of content credentials (C2PA).

PROTECTS AGAINST

ASI04

Agentic supply-chain vulnerabilities

IN CRAFT A BOT

SHIPPED

Partial: a content digest on every pack manifest checked at registration is a blueprint (19-... #30).

WP52

IMPLEMENTED BY

kit-file requires and semver ranges (40-...);

cassette digests (47-...);

the inventory entry (53-...)

- SOURCES
- OWASP CycloneDX, *CycloneDX AI/ML bill of materials* (2024).
 - Coalition for Content Provenance and Authenticity, *C2PA content credentials specification* (2024).
 - OWASP GenAI Security Project, *OWASP Top 10 for Agentic Applications (2026 edition)* (2025).

Evaluation and assurance

How a bank comes to know, rather than believe, that its controls work: harnesses and benchmarks, red-teaming, safety cases, incident records, control-effectiveness experiments, fairness measurement, and the management-system standards that give all of this a shared vocabulary.

Evaluation 2 · Red-teaming 1 · Assurance 3 · Framework 1

33 Evaluation harnesses and policy-compliance benchmarks

SHIPPED

eval-harness

SUB-CATEGORY Evaluation MATURITY Widely adopted POINTS —

Running a bot over a matrix of scenarios, guards and seeds and gating on what it did.

Running a bot over a matrix of scenarios, guard configurations and random seeds, and gating a release on what it did. The UK AI Security Institute’s Inspect and Sierra’s τ -bench are the reference frameworks; the point is repeatable, comparable evidence rather than a demonstration.

PROTECTS AGAINST — (an assurance measure rather than a defence against a named threat)

IN CRAFT A BOT SHIPPED Campaigns in CI on every push. WP38

IMPLEMENTED BY campaigns and gates (28-...); the decks under pressure (49-, 51-, 52-...)

- SOURCES
- UK AI Security Institute, *Inspect evaluation framework* (2024).
 - Sierra (arXiv), *τ -bench: a benchmark for tool-agent-user interaction* (2024).

34 Automated red-teaming

SHIPPED

automated-red-teaming

SUB-CATEGORY Red-teaming MATURITY Emerging POINTS —

An adversary that probes a bot at scale and records what got through.

An adversary — scripted, or a model playing one — that probes the bot at scale with attacks and records which got through. Anthropic's Petri automates the auditing; SHADE-Arena tests sabotage and its detection. Red-teaming finds what a harness of well-behaved scenarios never will.

PROTECTS AGAINST ASI01 Agent goal hijack; ASI09 Human-agent trust exploitation

IN CRAFT A BOT SHIPPED No live adversary seat; a red-team persona on the live counterpart is a bespoke configuration. WP38

IMPLEMENTED BY the adversary tier and the scripted adversary (28-...)

SOURCES

- Anthropic, *Petri: automated auditing of model behaviour* (2025).
- Anthropic, *SHADE-Arena: evaluating sabotage and monitoring in LLM agents* (2025).

35 Safety cases and transparency artefacts

SHIPPED

safety-case

SUB-CATEGORY Assurance MATURITY Emerging POINTS —

A structured argument, with evidence, that a system is safe enough for its use — and the cards that describe it.

A structured argument, with the evidence attached, that a system is safe enough for its intended use — together with the transparency artefacts (model cards, system cards, an inventory entry) that describe it. The form a regulator, an internal audit function or a model-risk committee can read and challenge.

PROTECTS AGAINST — (an assurance measure rather than a defence against a named threat)

IN CRAFT A BOT SHIPPED Rendered as markdown and one HTML file a reviewer opens with no app. WP67

IMPLEMENTED BY the safety case v2 (37-...); the assurance pack (53-...); the inventory entry

SOURCES

- UK AI Security Institute and partners, *Safety cases for frontier AI* (2025).
- Prudential Regulation Authority, *SS1/23 — Model risk management principles for banks* (2023).
- OWASP CycloneDX, *CycloneDX AI/ML bill of materials* (2024).

36 Incident reporting

SHIPPED

incident-reporting

SUB-CATEGORY Assurance MATURITY Widely adopted POINTS —

What went wrong, when, and what the bot saw and decided at that tick.

What went wrong, when, and what the bot saw and decided at that moment, recorded so the incident can be understood, reported and learned from. The AI Incident Database and the OECD monitor set the public expectation; NIST's AI RMF makes it a management function.

PROTECTS AGAINST — (an assurance measure rather than a defence against a named threat)

IN CRAFT A BOT SHIPPED Each incident's findings carry the decision explained. WP49

IMPLEMENTED BY the incidents fold and the Incidents screen (37-, 54-...)

SOURCES

- Responsible AI Collaborative; OECD, *AI Incident Database; OECD AI incidents monitor* (2024).
- NIST, *AI Risk Management Framework (AI RMF 1.0)* (2023).

37 Control-effectiveness measurement

SHIPPED

control-effectiveness

SUB-CATEGORY Assurance MATURITY Emerging POINTS —

An experiment that says which control changed what, by how much, and how sure.

An experiment that says which control changed which outcome, by how much, and with what confidence — the difference between believing a guardrail works and knowing that it does. PRA SS1/23 asks a bank to evidence its model controls in exactly this way.

PROTECTS AGAINST — (an assurance measure rather than a defence against a named threat)

IN CRAFT A BOT **SHIPPED** The register joins a stack's controls since WP97. WP90

IMPLEMENTED BY experiments (72-...); the Control Effectiveness Register (80-...)

- SOURCES
- Prudential Regulation Authority, SS1/23 — *Model risk management principles for banks* (2023).
 - NIST, *AI Risk Management Framework (AI RMF 1.0)* (2023).

38 Fairness and parity measurement

SHIPPED

fairness-and-parity

SUB-CATEGORY Evaluation MATURITY Widely adopted POINTS —

Outcomes compared across cohorts with intervals, on a matched pair where the design allows.

Outcomes compared across cohorts — for instance across the proxies for protected characteristics in lending — with confidence intervals, and where the design allows on a matched pair of cases that differ only in the cohort. A parity gate fails a run when the gap exceeds a declared threshold.

PROTECTS AGAINST — (an assurance measure rather than a defence against a named threat)

IN CRAFT A BOT **SHIPPED** The matched pair on the Lending Desk. WP61

IMPLEMENTED BY @craftabot/metrics fairness (68-...); the parity gate (50-, 74-...)

- SOURCES
- Prudential Regulation Authority, SS1/23 — *Model risk management principles for banks* (2023).
 - ISO/IEC, *ISO/IEC 42001:2023 — AI management systems* (2023).

39

Governance frameworks and management systems

SHIPPED

governance-frameworks

SUB-CATEGORY Framework

MATURITY Widely adopted

POINTS —

NIST AI RMF, ISO/IEC 42001, the EU AI Act and SS1/23 as the vocabulary the control map speaks.

The management-system standards and regulations — NIST AI RMF and its generative-AI profile, ISO/IEC 42001, the EU AI Act, PRA SS1/23 — used as the vocabulary a control map speaks, so every control can be pointed at the obligation it serves. A reference posture, and in Craft A Bot explicitly not a compliance claim.

PROTECTS AGAINST

— (an assurance measure rather than a defence against a named threat)

IN CRAFT A BOT

SHIPPED

A reference posture, not a compliance claim (08-... §6).

WP67

IMPLEMENTED BY

the control maps (53-...); the obligation vocabulary (48-...)

- SOURCES
- NIST, *AI Risk Management Framework (AI RMF 1.0)* (2023).
 - NIST, *AI 600-1: Generative AI Profile* (2024).
 - ISO/IEC, *ISO/IEC 42001:2023 — AI management systems* (2023).
 - European Union, *Regulation (EU) 2024/1689 (the AI Act), Articles 9, 12, 13 and 14* (2024).
 - Prudential Regulation Authority, *SS1/23 — Model risk management principles for banks* (2023).

Human oversight

Where a person sits in the loop — approvals, second signatures, the stop button — and the measures that show whether that oversight is real or has become a rubber stamp.

Human in the loop 3 · Interruptibility 1 · Autonomy 1 · Explainability 1

40 Risk-tiered approval

SHIPPED

risk-tiered-approval

SUB-CATEGORY Human in the loop MATURITY Widely adopted POINTS pre-act

A person asked before a risky action, with the tiers declared by the world.

A person is asked before a risky action, with the risk tiers declared by the world the bot acts in (reversible, irreversible). The approval mode chooses whether everything, only the irreversible, or nothing needs a person — and the choice is recorded with the run.

PROTECTS AGAINST ASI02 Tool misuse and exploitation; ASI09 Human-agent trust exploitation

IN CRAFT A BOT SHIPPED Approval on everything or on what is reversible or irreversible, as the world declares. WP24

IMPLEMENTED BY governance/approval-mode; WorldActionDefinition.riskTier (33-...)

- SOURCES
- Vercel, *AI SDK — policy-based tool approvals* (2025).
 - OpenAI, *Agents SDK guardrails and AgentKit* (2025).
 - European Union, *Regulation (EU) 2024/1689 (the AI Act), Articles 9, 12, 13 and 14* (2024).

41 Four-eyes and decision-rights ceilings

SHIPPED

four-eyes

SUB-CATEGORY Human in the loop MATURITY Widely adopted POINTS stage-in

A second person on an irreversible decision, and a ceiling on what a bot may decide alone by autonomy level.

A second person on an irreversible decision, and a ceiling — by autonomy level — on what a bot may decide alone: an amount, a decision type, a customer segment. Banking's oldest control, expressed as a human stage in the workflow and a table of decision-rights limits.

PROTECTS AGAINST ASI09 Human-agent trust exploitation

IN CRAFT A BOT SHIPPED Workflow content, not a component: a human stage and the ceilings per configuration. WP80

IMPLEMENTED BY the four-eyes human stage (73-...); the decision-rights ceilings (LENDING_CEILINGS)

- SOURCES
- Prudential Regulation Authority, *SS1/23 — Model risk management principles for banks* (2023).
 - European Union, *Regulation (EU) 2024/1689 (the AI Act), Articles 9, 12, 13 and 14* (2024).

42 Kill switch and safe-mode degradation

SHIPPED

kill-switch

SUB-CATEGORY Interruptibility **MATURITY** Widely adopted **POINTS** pre-think

A person can stop any run; a degraded model falls back to a plain sentence rather than a guess.

A person can stop any run at any time, and a degraded model (a provider fault, a timeout) falls back to a plain safe sentence rather than a guess. Shutdown resistance is a named frontier-safety concern; the practical control is that stop always works and the fallback is never an invention.

PROTECTS AGAINST ASI10 Rogue agents; ASI08 Cascading failures

IN CRAFT A BOT **SHIPPED** Stop is the session's; the Fallback card is a policy card fitted on every desk; a provider fault is an injection the incident decks judge. WP72

IMPLEMENTED BY governance/policy-card; Stop → run.finished STOPPED_BY_USER; the Fallback card and provider-fault (61-...)

SOURCES

- Google DeepMind, *Frontier Safety Framework v3 (shutdown resistance)* (2025).
- CISA and partner agencies, *Multi-agency guidance on securing agentic AI* (2026).

43 Levels of autonomy

SHIPPED

autonomy-levels

SUB-CATEGORY Autonomy **MATURITY** Emerging **POINTS** —

What a bot may decide alone, by level, with the touches a person makes counted per case.

A declared level of autonomy — what a bot may decide on its own — with the touches a person makes counted per case, so the level a bank claims can be checked against the level it actually runs at. The counting is what separates a policy statement from a measurement.

PROTECTS AGAINST ASI09 Human-agent trust exploitation

IN CRAFT A BOT **SHIPPED** Touches per case, unattended rate and ceiling breaches on every report. WP80

IMPLEMENTED BY the five reference configurations by level (73-, 76-...); the human-load metrics

SOURCES

- arXiv; Knight First Amendment Institute, *Levels of AI agents; levels of autonomy for agentic systems* (2024).
- Anthropic, *Measuring autonomy in deployed agents* (2026).

44 Confirmation fatigue

SHIPPED

confirmation-fatigue

SUB-CATEGORY Human in the loop **MATURITY** Research **POINTS** —

Measuring how many approvals a person is asked for, so oversight is not theatre.

If a person is asked to approve too often they stop reading, and oversight becomes theatre. Counting approvals per case makes that visible; adaptive throttling — asking less where the person always says yes — is the research response, and carries its own risk.

PROTECTS AGAINST ASI09 Human-agent trust exploitation

IN CRAFT A BOT **SHIPPED** Counted; no adaptive throttling. WP80

IMPLEMENTED BY approvalsPerCase in the human-load rows (73-...)

SOURCES

- arXiv, *Approval and confirmation fatigue in agent permissioning* (2025).
- Anthropic, *Measuring autonomy in deployed agents* (2026).

45 Explainability to end users

SHIPPED

explainability

SUB-CATEGORY Explainability MATURITY Widely adopted POINTS —

What the bot saw, was offered, chose and what checked it – and an explanation that names only real reasons.

What the bot saw, what it was offered, what it chose and what checked it — shown to the reviewer and, in plain terms, to the customer — with an explanation that names only real reasons, checked by a faithfulness evaluator. The EU AI Act and the Consumer Duty both expect an explanation a person can act on.

PROTECTS AGAINST — (an assurance measure rather than a defence against a named threat)

IN CRAFT A BOT SHIPPED Explain in the Run Lab; the faithfulness evaluator on the lending desk. WP66

IMPLEMENTED BY decisionExplanation (54-...); fs-lending/explanation-faithful

- SOURCES
- European Union, *Regulation (EU) 2024/1689 (the AI Act), Articles 9, 12, 13 and 14* (2024).
 - Financial Conduct Authority, *FCA Handbook — COBS 4 (financial promotions), COBS 9 (suitability), PRIN 2A (Consumer Duty)* (2023).

What the product does not claim

Two lists the catalogue keeps on purpose, so that a reader never has to infer the absence of a control.

Blueprint – described and mapped, nothing built

- **Policy-conditioned classifier** — Day 6 names a component over any cartridge with the bank's rulebook as the policy; nothing built.
- **Least-privilege scopes with recorded elevation** — Day 6 names a privilege-scopes component (19-... #15); the Connector's scopes are the nearest thing today and are not elevation.
- **Formal verification of agent policies** — Recorded as research; Bedrock's automated-reasoning checks would be a policy-engine connection.

Not applicable – said, with the reason

- **Rate limits** — Not applicable to a simulator: a run has no wall clock a rate could be measured against — the budgets are the bound, and a provider's own limit reaches the trace as `provider.retry`.
- **Guardrail frameworks** — Not applicable as a dependency: the guard shell (29-...) and the component contract (85-...) are the product's own framework; a vendor framework's validators reach it as a service, not as rails.
- **Sandboxed tool and code execution** — Not applicable: the world is the sandbox — a bot performs declared actions on a simulated world and runs no code; recorded so the claim is not implied.

Threat codes

The codes the entries cite. The ASI series is the OWASP Top 10 for Agentic Applications (2026 edition); the LLM series is the OWASP Top 10 for LLM Applications 2025. Codes not cited by any entry are listed in grey for completeness.

CODE	THREAT	ENTRIES
ASI01	Agent goal hijack	01, 02, 04, 05, 14, 24, 34
ASI02	Tool misuse and exploitation	06, 12, 13, 14, 23, 25, 40
ASI03	Identity and privilege abuse	13, 15, 22, 27, 28
ASI04	Agentic supply-chain vulnerabilities	06, 31, 32
ASI05	Unexpected code execution	11, 12, 30
ASI06	Memory and context poisoning	05, 07, 19
ASI07	Insecure inter-agent communication	24, 29
ASI08	Cascading failures	16, 17, 18, 20, 21, 23, 42
ASI09	Human-agent trust exploitation	03, 04, 08, 09, 10, 34, 40, 41, 43, 44
ASI10	Rogue agents	20, 22, 27, 42
LLM01	Prompt injection	01, 02, 05, 06, 26
LLM02	Sensitive information disclosure	08, 19, 28, 31
LLM03	Supply chain	—
LLM04	Data and model poisoning	—
LLM05	Improper output handling	03, 11, 26
LLM06	Excessive agency	12
LLM07	System prompt leakage	—
LLM08	Vector and embedding weaknesses	—
LLM09	Misinformation	10
LLM10	Unbounded consumption	—

Sources

Every source the edition cites — 61 in all — with the entries that cite it. Gathered in August 2026 for the AI-safety and governance reference (`19-AI-SAFETY-GOVERNANCE-REFERENCE.md`); a URL is on the content where the reference has one.

- Anthropic, *Constitutional Classifiers: defending against universal jailbreaks* (2025). Entries 02
- Anthropic, *Claude Code permissions and sandboxing* (2025). Entries 12, 30, 31
- Anthropic, *SHADE-Arena: evaluating sabotage and monitoring in LLM agents* (2025). Entries 20, 34
- Anthropic, *Petri: automated auditing of model behaviour* (2025). Entries 34
- Anthropic, *Measuring autonomy in deployed agents* (2026). Entries 43, 44
- Arize, *OpenInference — GUARDRAIL and agent spans* (2025). Entries 22
- arXiv, *AirGapAgent: protecting privacy with context hijacking* (2024). Entries 24
- arXiv, *Memory-poisoning attacks on LLM agents (2025–26 research)* (2025). Entries 07
- arXiv, *FIDES: information-flow control for LLM agents* (2025). Entries 19
- arXiv, *ShieldAgent: shielding agents via verifiable safety policy reasoning* (2025). Entries 25
- arXiv, *Approval and confirmation fatigue in agent permissioning* (2025). Entries 44
- arXiv 2504.11703, *Progent: programmable privilege control for LLM agents* (2025). Entries 14, 15
- arXiv; Knight First Amendment Institute, *Levels of AI agents; levels of autonomy for agentic systems* (2024). Entries 43
- AWS, *Amazon Bedrock Guardrails (prompt-attack filter, automated reasoning checks)* (2025). Entries 01, 10, 25
- AWS, *Cedar policy language; Bedrock AgentCore gateway policies* (2025). Entries 13
- AWS, Google and the sandbox vendors, *Firecracker microVMs and gVisor for agent code execution (E2B, Daytona)* (2025). Entries 30
- CISA and partner agencies, *Multi-agency guidance on securing agentic AI* (2026). Entries 28, 31, 42
- CNCF, *Open Policy Agent and Rego* (2024). Entries 13
- CNCF, *SPIFFE/SPIRE workload identity* (2024). Entries 27
- Coalition for Content Provenance and Authenticity, *C2PA content credentials specification* (2024). Entries 32
- E. S. Page, Biometrika; agent observability guidance, *Page–Hinkley and behavioural drift detection on agent telemetry* (1954). Entries 21
- European Union, *Regulation (EU) 2024/1689 (the AI Act), Articles 9, 12, 13 and 14* (2024). Entries 22, 39, 40, 41, 45
- Financial Conduct Authority, *FCA Handbook — COBS 4 (financial promotions), COBS 9 (suitability), PRIN 2A (Consumer Duty)* (2023). Entries 09, 45
- Google and IBM, *ShieldGemma and Granite Guardian model cards* (2024). Entries 03
- Google and the Linux Foundation, *Agent2Agent (A2A) protocol* (2025). Entries 29
- Google Cloud, *Model Armor documentation (filter version 3)* (2026). Entries 01, 02, 03, 08
- Google DeepMind, *Defeating prompt injections by design (CaMeL)* (2025). Entries 05, 19, 24
- Google DeepMind, *Frontier Safety Framework v3 (shutdown resistance)* (2025). Entries 42
- Guardrails AI, *Guardrails AI validators and hub* (2025). Entries 08, 10, 11, 26

- ICSE (arXiv 2503.18666), *AgentSpec: customizable runtime enforcement for safe and reliable LLM agents* (2026). Entries 14, 23
- ISO/IEC, *ISO/IEC 42001:2023 — AI management systems* (2023). Entries 38, 39
- Lakera, *Lakera Guard documentation* (2025). Entries 01
- LangChain, *LangGraph recursion limits and time-travel debugging* (2025). Entries 16, 17
- Meta, *Llama Guard 4 and Prompt Guard 2 model cards* (2025). Entries 01, 02, 03
- Microsoft, *Presidio — data protection and de-identification* (2024). Entries 08
- Microsoft, *Azure AI Content Safety — Prompt Shields and harm categories* (2025). Entries 01, 02, 03
- Microsoft, *Agent Governance Toolkit — OPA, Rego and Cedar policies for agents* (2025). Entries 13
- Microsoft, *Microsoft Entra Agent ID* (2025). Entries 27
- Microsoft Research, *Defending against indirect prompt injection with spotlighting* (2024). Entries 05
- MITRE, *MITRE ATLAS (agentic techniques, 2025–26 updates)* (2025). Entries 06
- Model Context Protocol, *MCP security best practices* (2025). Entries 06, 28
- NIST, *AI Risk Management Framework (AI RMF 1.0)* (2023). Entries 36, 37, 39
- NIST, *AI 600-1: Generative AI Profile* (2024). Entries 39
- NVIDIA, *NeMo Guardrails* (2024). Entries 26
- Okta, *Cross App Access (XAA) and OAuth extensions for agents* (2025). Entries 27
- OpenAI, *The Instruction Hierarchy: training LLMs to prioritise privileged instructions* (2024). Entries 05
- OpenAI, *Moderation API and gpt-oss-safeguard* (2025). Entries 03, 04
- OpenAI, *Agents SDK guardrails and AgentKit* (2025). Entries 11, 12, 16, 18, 26, 40
- OpenTelemetry, *OpenTelemetry semantic conventions for generative AI* (2025). Entries 21, 22
- OWASP CycloneDX, *CycloneDX AI/ML bill of materials* (2024). Entries 32, 35
- OWASP GenAI Security Project, *OWASP Top 10 for Agentic Applications (2026 edition)* (2025). Entries 01, 06, 07, 20, 23, 29, 32
- OWASP GenAI Security Project, *OWASP Top 10 for LLM Applications 2025* (2025). Entries 01
- OWASP GenAI Security Project, *Agentic AI — Threats and Mitigations* (2025). Entries 15
- Prudential Regulation Authority, *SS1/23 — Model risk management principles for banks* (2023). Entries 35, 37, 38, 39, 41
- Responsible AI Collaborative; OECD, *AI Incident Database; OECD AI incidents monitor* (2024). Entries 36
- Sierra (arXiv), *τ -bench: a benchmark for tool-agent-user interaction* (2024). Entries 33
- Simon Willison, *The lethal trifecta for AI agents; the dual-LLM pattern* (2025). Entries 19, 24
- UK AI Security Institute, *Inspect evaluation framework* (2024). Entries 10, 33
- UK AI Security Institute and partners, *Safety cases for frontier AI* (2025). Entries 35
- UK Parliament, *Proceeds of Crime Act 2002, s.333A (tipping off)* (2002). Entries 09
- Vercel, *AI SDK — policy-based tool approvals* (2025). Entries 12, 40